universitas
MALIKUSSALEH

# Pemrograman Berorientasi Objek (OOP)

Pemrograman Prosedural vs OOP, Kelas, Objek, dan Abstraksi

Ir. Shaki Saptiadi Putra, S.T., M.Eng., IPP.

Program Studi Teknik Mesin
Fakultas Teknik
Universitas Malikussaleh

# Agenda Perkuliahan

Paradigma Prosedural vs OOP

Kelas, Objek, dan Atribut

Pilar OOP 1: Abstraction (Abstraksi)

# Evolusi Cara Berpikir Kita

Sejauh ini, kita menulis program dengan cara yang lurus: mendefinisikan variabel, membuat percabangan, perulangan, dan menyusun fungsi-fungsi. Ini disebut **Pemrograman Prosedural**.

Paradigma prosedural melihat program sebagai **urutan instruksi (tindakan)** yang beroperasi pada data yang terpisah.

Namun, ketika kita mulai membangun sistem kompleks seperti simulasi dinamis kendaraan listrik (EV) di mana ada baterai, motor DC, dan sistem pengereman, cara prosedural menjadi sangat berantakan dan sulit dikelola.

# Paradigma Pemrograman Berorientasi Objek (OOP)

**Object-Oriented Programming (OOP)** mengubah cara pandang kita 180 derajat. Daripada memikirkan urutan tindakan, kita memodelkan program sebagai **kumpulan objek fisis/konseptual** yang saling berinteraksi.

**Cara Prosedural:**
Fokus pada **Kata Kerja (Aksi)**

- `hitung_torsi()`
- `baca_sensor_suhu()`
- Data suhu dan fungsi hitung tersebar dan terpisah.

**Cara OOP:**
Fokus pada **Kata Benda (Objek)**

- Objek: `MotorListrik`
- Objek ini "memiliki" data (suhu, daya) dan "tahu cara" menghitung torsinya sendiri.

# Kelas (*Class*): Cetak Biru (Blueprint)

Mari gunakan bahasa bengkel. Jika Anda ingin membuat sebuah komponen roda gigi kustom, Anda akan menggambar desainnya terlebih dahulu di AutoCAD.

**Desain (Gambar Teknik)** inilah yang disebut sebagai **Kelas (*Class*)**.

## Definisi Kelas

Sebuah cetak biru (blueprint) atau cetakan abstrak yang mendefinisikan karakteristik (atribut) dan perilaku (metode) dari suatu entitas. Kelas belum memiliki wujud fisik di memori komputer.

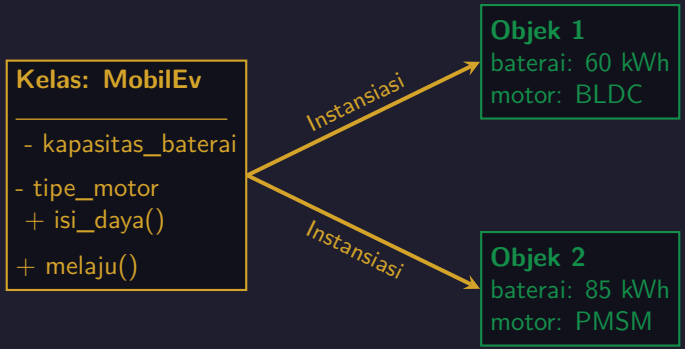
# Objek (*Object*): Wujud Nyata (Instance)

Setelah gambar teknik selesai, operator CNC akan mengeksekusi mesin untuk membuat wujud fisik roda gigi dari baja pejal.

**Benda fisik nyata** hasil cetakan itulah yang disebut **Objek (*Object*)**.

Dari satu Kelas (Blueprint), kita bisa mencetak puluhan hingga ribuan Objek (Instance) yang saling independen di dalam memori komputer.

# Visualisasi Kelas ke Objek



# Atribut (*Attribute*) dan Metode (*Method*)

Sebuah objek memiliki dua komponen utama penyusunnya:

**1. Atribut (State/Data):** Variabel yang terikat ke dalam objek tersebut. Ini mewakili "sifat" atau "status" objek.

- Contoh pada `MobilEv`: `warna`, `status_baterai`, `suhu_motor`.

**2. Metode (Behavior/Tingkah Laku):** Fungsi yang terikat ke dalam objek. Ini mewakili "apa yang bisa dilakukan" oleh objek.

- Contoh pada `MobilEv`: `akselerasi()`, `pengereman_regeneratif()`.

# Struktur Sintaksis Kelas (Python)

Mari kita lihat bagaimana konsep Blueprint ini dituliskan dalam sintaksis Python riil:

```
class MotorDC:
    # Constructor:  Fungsi yg dipanggil pertama kali saat objek dicetak
    def __init__(self, voltase_nominal, resistansi):
        self.volt = voltase_nominal # Atribut 1
        self.res = resistansi # Atribut 2
    # Method:  Tingkah laku motor
    def hitung_arus_stall(self):
        return self.volt / self.res
```

*Catatan: Kata kunci `self` digunakan agar metode/objek mengenali datanya sendiri, tidak tertukar dengan objek MotorDC lain di memori.*

# Proses Instansiasi (Mencetak Objek)

Setelah cetakan (Kelas) dibuat, kita akan mencetak dua unit motor fisik (Objek) ke dalam memori dengan spesifikasi berbeda:

```
 # Mencetak Objek 1 (Motor kecil)
motor_wiper = MotorDC(12, 0.5)
# Mencetak Objek 2 (Motor traksi KMHE)
motor_penggerak = MotorDC(48, 0.02)
# Memanggil metode masing-masing objek secara independen
print( motor_wiper.hitung_arus_stall() )  # Output:  24.0
print( motor_penggerak.hitung_arus_stall() )  # Output:  2400.0
```

# Mengapa Kita Butuh Pilar?

Pemrograman Berorientasi Objek bukan sekadar mengelompokkan variabel ke dalam kelas. OOP dibangun di atas 4 pilar fundamental (Pilar Desain Sistem):

1. **Abstraction** (Abstraksi)
2. Encapsulation (Enkapsulasi / Pembungkusan)
3. Inheritance (Pewarisan)
4. Polymorphism (Polimorfisme)

Hari ini, kita akan membedah pilar pertama yang menjadi landasan desain: **Abstraksi**.

# Konsep Abstraction (Abstraksi)

## Definisi Abstraksi

Abstraksi adalah proses menyembunyikan detail kompleksitas internal sebuah sistem dari pengguna (*user* atau *programmer* lain), dan hanya menampilkan antarmuka fungsional yang penting dan mudah dipahami.

Analogi di Teknik Mesin: **Pedal Gas Mobil**. Saat Anda menginjak pedal gas, Anda tidak perlu tahu rasio campuran bahan bakar-udara yang diinjeksikan ECU, atau bukaan katup gas secara persentase. Anda hanya tahu: "Injak = Akselerasi".

# Abstraksi dalam Kode: Menyembunyikan Kompleksitas

Dalam kode program simulasi, saat kita memanggil metode `motor.akselerasi()`, kita mengabstraksikan persamaan diferensial Runge-Kutta orde-4 atau algoritma PID dari pandangan programmer utama.

```
 # Programmer utama HANYA melihat ini:
mobil_kmhe.akselerasi(target_kecepatan=40)

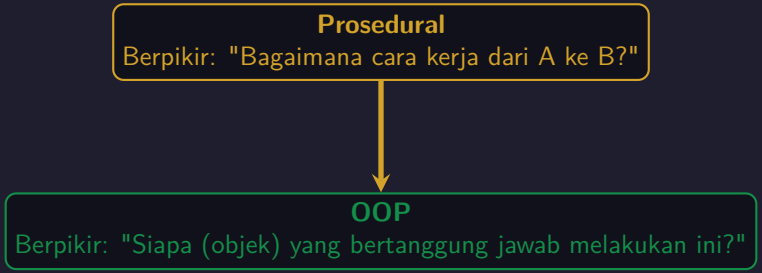
# Padahal di dalam kelas (Abstracted), terjadi kalkulasi berat:
def akselerasi(self, target):
    err = target - self.sensor_rpm()
    sinyal_kontrol = self.Kp*err + self.Ki*integral(err)
    self.inverter.kirim_pwm(sinyal_kontrol)
```

# Pentingnya Abstraksi dalam Kolaborasi Tim

Bagi mahasiswa teknik yang kelak mengembangkan perangkat lunak sistem tertanam (*embedded system*) atau sistem IoT secara berkelompok (seperti tim mobil listrik), abstraksi sangat vital:

- **Fokus pada Solusi Global:** Programmer antarmuka visual (UI) tidak perlu pusing memikirkan logika bitwise sensor. Mereka cukup memanggil `sensor.baca_suhu()`.
- **Pengurangan Kesalahan:** Dengan menyembunyikan kerumitan matematis di dalam "kotak hitam" (metode kelas), risiko komponen lain merusak variabel internal menjadi sangat kecil.
- **Modularitas:** Jika algoritma kontrol PID di dalam kelas diganti menjadi *Machine Learning*, kode programmer antarmuka di luar kelas tidak perlu diubah sama sekali.

# Kesimpulan: "Berpikir seperti Desainer Sistem"




Transisi ke Pemrograman Berorientasi Objek menuntut Anda naik level: dari sekadar "tukang ketik instruksi", menjadi seorang **Arsitek Perangkat Lunak** yang mendesain interaksi antar komponen virtual secara logis, aman, dan elegan (terabstraksi).

# Terima Kasih

Ada Pertanyaan?



universitas
MALIKUSSALEH